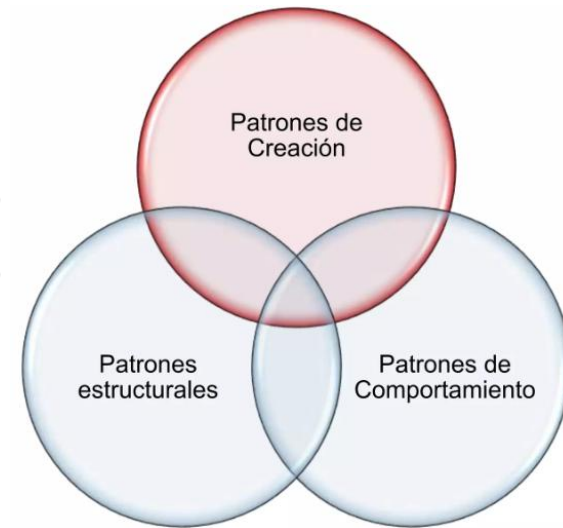


PDS-CREACIONALES: SINGLETON

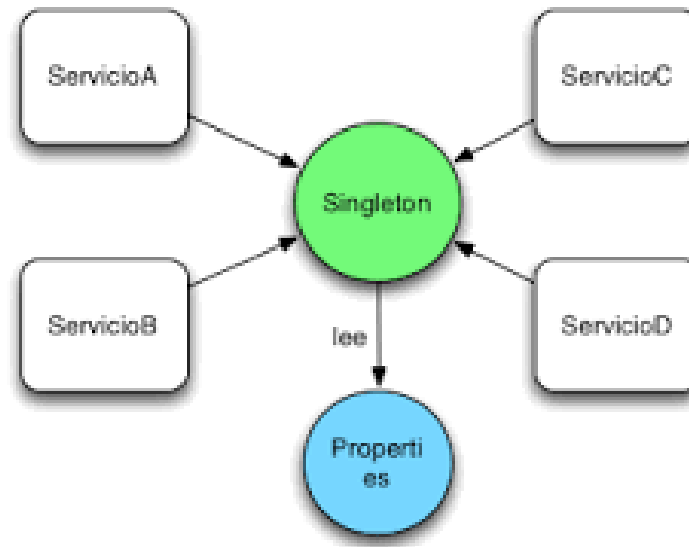


ING. ZOILA RUIZ, PHD

SINGLETON

El patrón Singleton o Instancia Única permite restringir la creación de objetos, garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

Este patrón se utiliza cuando se necesita exactamente una instancia de una clase para coordinar acciones en todo el sistema. Por ejemplo, para un objeto de configuración, una conexión a base de datos, o un logger.



SINGLETON

Contexto: Necesita asegurar que solo exista una instancia de una clase y proporcionar un punto de acceso global a ella.

Problema: ¿Cómo restringir la creación de múltiples instancias de una clase que representa un recurso único (por ejemplo, configuración, logger, base de datos)?

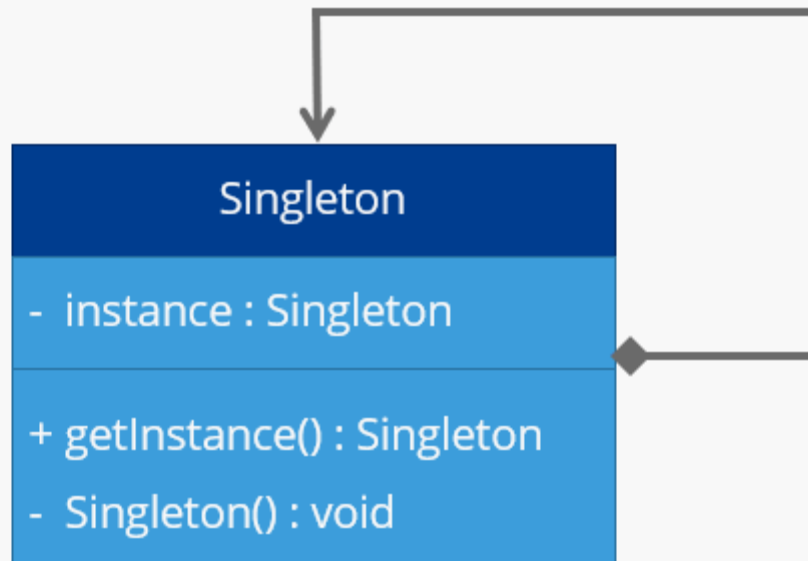
Solución: Crear una clase con:

- Un constructor privado
- Una instancia estática privada de sí misma
- Un método público que devuelve la instancia única (lazy o eager)

CONSECUENCIAS POSITIVAS	CONSECUENCIAS NEGATIVAS
Control de acceso a una única instancia	Difícil de testear (oculta dependencia)
Reduce uso de recursos compartidos	Puede generar acoplamiento global
	Problemas de concurrencia

SINGLETON

Diagrama UML: patrón singleton



La idea principal para su implementación es:

- ❖ **Ocultar el constructor** (haciendo private el constructor).
- ❖ Crear una **instancia estática** de la clase dentro de ella misma.
- ❖ Proporcionar un **método estático** que devuelva la única instancia.

SINGLETON

Usos:

- ❖ Antivirus, conexión a base de datos, Impresoras o hardware, Administrador de ventanas
- ❖ Cuando necesitamos controlar el acceso a la instancia
- ❖ El control debe de permanecer en la clase instancia y no en otro mecanismo
- ❖ Crear una excepción si se intenta más de una vez la clase
- ❖ Todos los componentes de negocios en Spring son Singleton

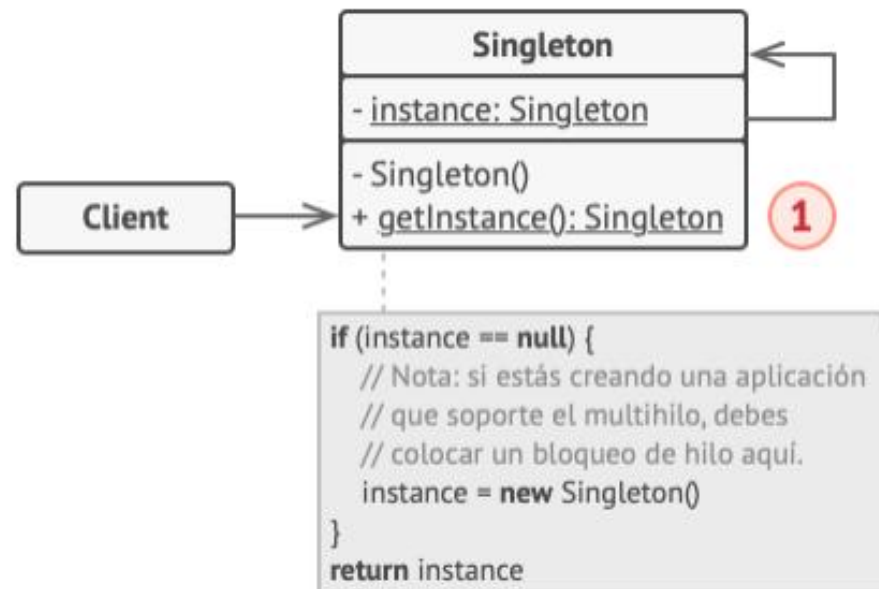
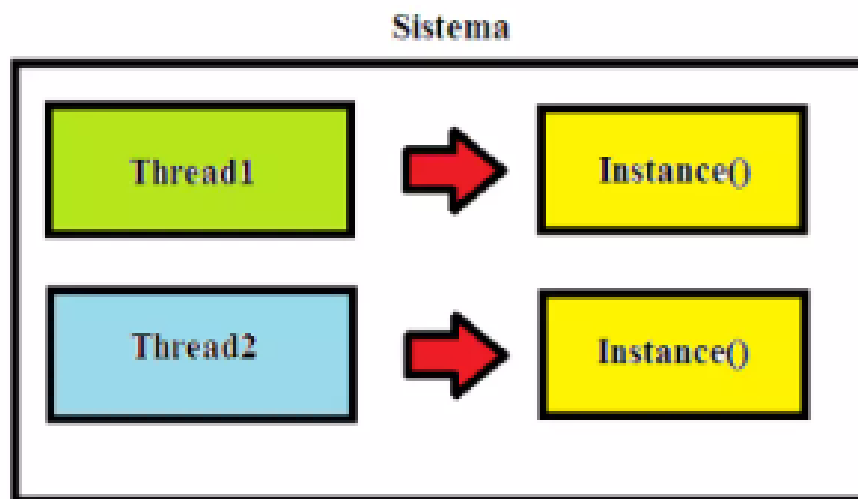
El patrón Singleton es útil cuando se necesita un control estricto sobre la creación de instancias de una clase.

Es fundamental implementar correctamente el patrón para evitar problemas de concurrencia y acoplamiento.

SINGLETON

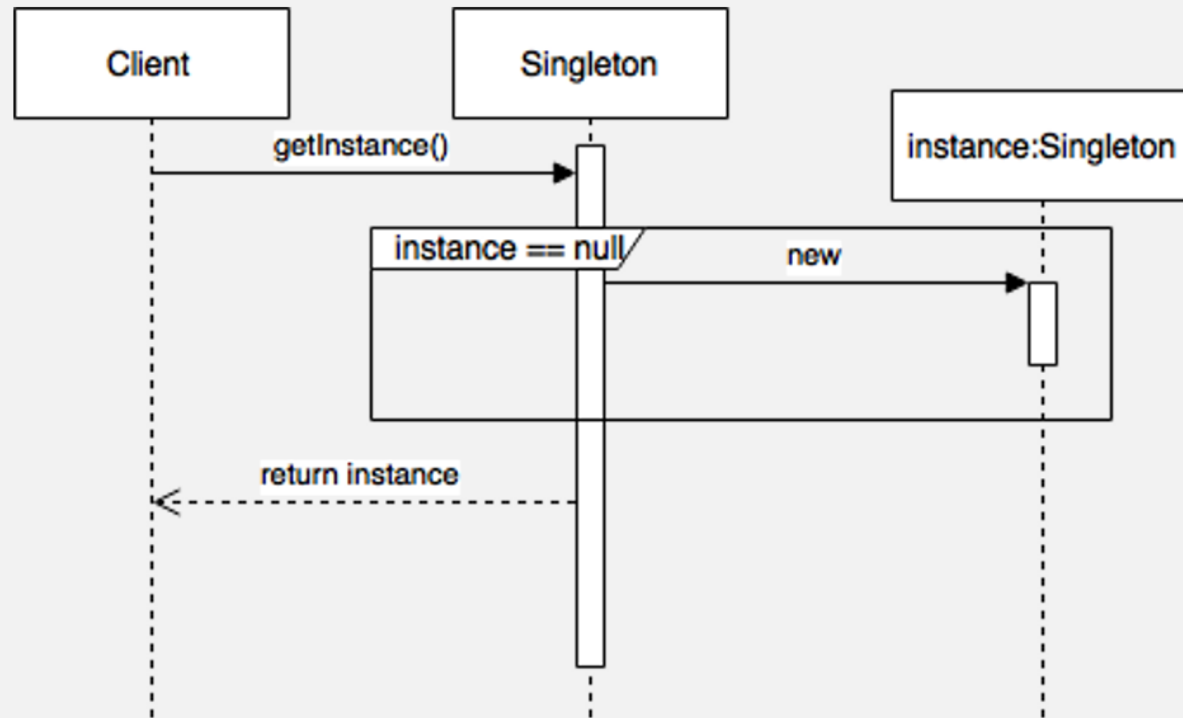
Problemas en un ambiente multihilo:

En ambiente de un solo hilo (single-thread) su implementación es simple. En ambientes multihilo (multi-thread), existen serios problemas dado que, por sincronización y concurrencia, puede crearse más de una instancia del miembro instance.



SINGLETON

Singleton pattern – Diagram of sequence



- ❖ Garantiza que exista sola instancia de una clase.
- ❖ Crea un punto global de acceso para ese objeto.

SINGLETON



SingletonDatabase

- ❑ instance: SingletonDatabase
- ❑ DB_URL: String
- ❑ connection: Connection
- SingletonDatabase()
 - getInstance(): SingletonDatabase
 - query(sql: String, params: Object[]): Object
 - querySelect(sql: String, params: Object[]): List<Map<String, Object>>
 - queryUpdate(sql: String, params: Object[]): int
 - close(): void

SINGLETON

SingletonDatabasePro

❑ instance: SingletonDatabasePro

❑ connection: Connection

■ SingletonDatabasePro()

■ loadDatabaseUrl(): String

● getInstance(): SingletonDatabasePro

● query(sql: String, params: Object[]): Object

● querySelect(sql: String, params: Object[]): List<Map<String, Object>>

● queryUpdate(sql: String, params: Object[]): int

● close(): void